

Rajalakshmi Engineering College

Name: Sheik Abdulla
Email: 240701493@rajalakshmi.edu.in
Roll no: 240701493
Phone: 6379676782
Branch: REC
Department: CSE - Section 9
Batch: 2028
Degree: B.E - CSE

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 8_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

In an online shopping cart system, users can apply coupon codes during checkout to avail of discounts. However, to ensure the validity and security of coupon codes, the system enforces specific rules for their format. Your task is to implement a Java program named `CouponCodeValidator` that takes user input for a coupon code and validates it according to the specified rules.

Rules for Valid Coupon Code:

The coupon code must consist of exactly 10 characters. The coupon code must contain at least one alphabet (uppercase or lowercase) and at least one digit (0-9). Special characters are not allowed in the coupon code.

Implement a custom exception, `InvalidCouponException`, to handle cases where the entered coupon code does not meet the specified criteria.

Input Format

The input consists of a string *s*, representing the coupon code.

Output Format

The output is displayed in the following format:

If the entered coupon code meets the specified criteria, the program outputs

"Coupon code applied successfully!"

If the entered coupon code has less than or more than 10 characters it outputs

"Error: Invalid coupon code length. It must be exactly 10 characters."

If the entered coupon code contains only numeric or only alphabets it outputs

"Error: Invalid coupon code format. It must contain at least one alphabet and one digit."

If the entered coupon code contains special characters it outputs

"Error: Coupon code should not contain special characters."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: ABCD123456

Output: Coupon code applied successfully!

Answer

```
import java.util.Scanner;
```

```
// Custom Exception
```

```
class InvalidCouponException extends Exception {  
    public InvalidCouponException(String message) {  
        super(message);  
    }  
}
```

```
}
```

```
public class Main {  
    // Method to validate the coupon code  
    public static void validateCoupon(String code) throws InvalidCouponException  
    {  
        if (code.length() != 10) {  
            throw new InvalidCouponException("Error: Invalid coupon code length. It  
must be exactly 10 characters.");  
        }  
  
        boolean hasLetter = false;  
        boolean hasDigit = false;  
        for (char c : code.toCharArray()) {  
            if (!Character.isLetterOrDigit(c)) {  
                throw new InvalidCouponException("Error: Coupon code should not  
contain special characters.");  
            }  
            if (Character.isLetter(c)) hasLetter = true;  
            if (Character.isDigit(c)) hasDigit = true;  
        }  
  
        if (!hasLetter || !hasDigit) {  
            throw new InvalidCouponException("Error: Invalid coupon code format. It  
must contain at least one alphabet and one digit.");  
        }  
  
        // Passed all checks  
        System.out.println("Coupon code applied successfully!");  
    }  
  
    // Main method to run the program  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        String couponCode = scanner.nextLine();  
  
        try {  
            validateCoupon(couponCode);  
        } catch (InvalidCouponException e) {  
            System.out.println(e.getMessage());  
        }  
    }  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Tim was tasked with creating a user profile system that validates the user's date of birth input. The system should throw a custom exception, `InvalidDateOfBirthException`, if the date is not in the specified format "dd-mm-yyyy" or if it represents an invalid calendar date.

The main method takes user input, validates the date of birth, and prints whether it is valid or not.

Input Format

The input consists of a string, representing the date of birth of the user.

Output Format

The output displays one of the following results:

If the entered date of birth is valid according to the specified format, the program prints:

"[Date] is a valid date of birth"

If the entered date of birth is not valid according to the specified format, the program prints:

"Invalid date: [Date]"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 01-01-2000

Output: 01-01-2000 is a valid date of birth

Answer

```
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Scanner;

// Custom Exception
class InvalidDateOfBirthException extends Exception {
    public InvalidDateOfBirthException(String message) {
        super(message);
    }
}

public class Main {

    // Method to validate date format and validity
    public static void validateDateOfBirth(String dob) throws
    InvalidDateOfBirthException {
        SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");
        sdf.setLenient(false); // This ensures strict parsing (e.g., 31-02-2022 will be
        invalid)

        try {
            sdf.parse(dob); // Try parsing the date
            System.out.println(dob + " is a valid date of birth");
        } catch (ParseException e) {
            throw new InvalidDateOfBirthException("Invalid date: " + dob);
        }
    }

    // Main method
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String dob = scanner.nextLine();

        try {
            validateDateOfBirth(dob);
        } catch (InvalidDateOfBirthException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

}

Status : Correct

Marks : 10/10

3. Problem Statement

Hemanth is designing a banking system for XYZ Bank. The system should allow customers to perform deposit, withdrawal, and balance inquiry operations. Implement exception handling for scenarios involving invalid transaction amounts or insufficient funds.

Create two custom exception classes, `InvalidAmountException` and `InsufficientFundsException`, both extending the `Exception` class. Throw an `InvalidAmountException` with a message if the deposit amount is less than or equal to zero. Throw an `InsufficientFundsException` if the withdrawal amount is greater than the available balance. Deduct the withdrawal amount from the balance if the withdrawal is successful.

Assist Hemanth in designing the program.

Input Format

The first line of input consists of a double value `B`, representing the initial balance.

The second line consists of a double value `D`, representing the deposit amount.

The third line consists of a double value `W`, representing the withdrawal amount.

Output Format

If the withdrawal is successful, print the amount withdrawn and the current balance, rounded off to one decimal place.

If an `InvalidAmountException` occurs, print "Error: [D] is not valid".

If an `InsufficientFundsException` occurs, print "Error: Insufficient funds".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1050.1

270.2

150.3

Output: Amount Withdrawn: 150.3

Current Balance: 1170.0

Answer

```
import java.util.Scanner;
```

```
// Custom Exception for Invalid Amount
```

```
class InvalidAmountException extends Exception {  
    public InvalidAmountException(String message) {  
        super(message);  
    }  
}
```

```
// Custom Exception for Insufficient Funds
```

```
class InsufficientFundsException extends Exception {  
    public InsufficientFundsException(String message) {  
        super(message);  
    }  
}
```

```
public class Main {
```

```
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);
```

```
        // Read inputs
```

```
        double balance = scanner.nextDouble();
```

```
        double deposit = scanner.nextDouble();
```

```
        double withdrawal = scanner.nextDouble();
```

```
        try {
```

```
            // Validate deposit amount
```

```
            if (deposit <= 0) {
```

```
                throw new InvalidAmountException("Error: " + deposit + " is not valid");
```

```
            }
```

```
            // Add deposit to balance
```



```

        balance += deposit;

        // Validate withdrawal
        if (withdrawal > balance) {
            throw new InsufficientFundsException("Error: Insufficient funds");
        }

        // Deduct withdrawal
        balance -= withdrawal;

        // Print successful withdrawal message
        System.out.printf("Amount Withdrawn: %.1f Current Balance: %.1f%n",
            withdrawal, balance);
    } catch (InvalidAmountException | InsufficientFundsException e) {
        // Print the exception message
        System.out.println(e.getMessage());
    }
}
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Theo is trying to update his payment information on a subscription-based streaming service. To proceed, the system requires Theo to provide a valid credit card number consisting of 16 digits. However, Theo wants to make sure that the credit card number he enters meets the specified criteria with proper exception handling.

The credit card number must consist of exactly 16 digits. If the entered credit card number does not meet the specified criteria, the program should throw a custom exception, `InvalidCreditCardException`, and provide Theo with specific error messages: If the length of the credit card number is not 16 digits, the exception message should be: "Invalid credit card number length." If the credit card number contains non-numeric characters, the exception message should be: "Invalid credit card number format."

Implement a custom exception, InvalidCreditCardException, to fulfill Theo's requirements and keep his payment information secure.

Input Format

The input consists of a string value 's', consisting of the 16-digit credit card number.

Output Format

The output is displayed in the following format:

If the entered credit card number is valid, the program should output a success message:

"Payment information updated successfully!"

If the entered credit card has more than 16 digits or less than 16 digits it displays

"Error: Invalid credit card number length."

If the entered 16-digit credit card has non-integers it displays

"Error: Invalid credit card number format."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1234567890123456

Output: Payment information updated successfully!

Answer

```
import java.util.Scanner;
```

```
// Custom Exception
```

```
class InvalidCreditCardException extends Exception {  
    public InvalidCreditCardException(String message) {  
        super(message);  
    }  
}
```

```

}

public class Main{

    // Validation Method
    public static void validateCreditCard(String cardNumber) throws
    InvalidCreditCardException {
        // Check for length
        if (cardNumber.length() != 16) {
            throw new InvalidCreditCardException("Invalid credit card number
length.");
        }

        // Check if all characters are digits
        if (!cardNumber.matches("\\d{16}")) {
            throw new InvalidCreditCardException("Invalid credit card number
format.");
        }

        // If both checks pass
        System.out.println("Payment information updated successfully!");
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String cardNumber = scanner.nextLine();

        try {
            validateCreditCard(cardNumber);
        } catch (InvalidCreditCardException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

Status : Correct

Marks : 10/10